

Requerimientos

Canvas (lienzo)

Para la visualización de los planos se reviso un componente llamado BlueprintCanvas, encargado de representar gráficamente los puntos que conforman cada blueprint. Este componente utiliza el elemento HTML <canvas> para dibujar los planos de forma dinámica dentro de la aplicación.

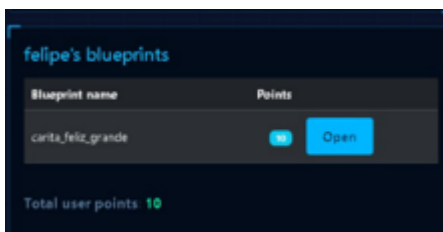
El lienzo fue configurado con dimensiones de 520 × 360 píxeles, lo que permite visualizar adecuadamente los planos sin ocupar toda la pantalla. Además, mediante el uso de React Hooks (useRef y useEffect), el componente obtiene el contexto del canvas y redibuja automáticamente el contenido cuando cambian los puntos del blueprint.



Listar los planos de un autor

En esta etapa se revisa la funcionalidad que permite consultar los planos (blueprints) asociados a un autor específico. Para ello, la interfaz permite ingresar el nombre del autor y realizar una consulta al servicio correspondiente.

Una vez obtenidos los datos, los resultados se muestran en una tabla dentro de la aplicación. Cada fila de la tabla representa un plano y contiene la siguiente información:



Seleccionar un plano y graficarlo

En este punto revisa la funcionalidad para visualizar un plano seleccionado. Cuando el usuario presiona el botón Open en la lista de planos, se selecciona el blueprint correspondiente y se actualiza el nombre del plano actual. Luego se obtienen sus puntos (coordenadas) y estos se envían al componente de Canvas, donde se dibujan consecutivamente los segmentos de recta que forman el plano y se marcan visualmente cada uno de los puntos. De esta forma el usuario puede ver la representación gráfica del plano seleccionado.

```
import { useEffect, useRef, useState } from 'react'

export default function BlueprintCanvas({ points = [], width = 520, height = 360, onAddPoint }) {
  const ref = useRef(null)
  const [mousePos, setMousePos] = useState(null)

  useEffect(() => {
    const canvas = ref.current
    if (!canvas) return

    const ctx = canvas.getContext('2d')

    // limpiar canvas
    ctx.clearRect(0, 0, canvas.width, canvas.height)

    // fondo
    ctx.fillStyle = '#0b1220'
    ctx.fillRect(0, 0, canvas.width, canvas.height)

    // GRID
    ctx.strokeStyle = 'rgba(148,163,184,0.12)'
    ctx.lineWidth = 1

    for (let x = 0; x < canvas.width; x += 40) {
      ctx.beginPath()
      ctx.moveTo(x, 0)
      ctx.lineTo(x, canvas.height)
      ctx.stroke()
    }

    for (let y = 0; y < canvas.height; y += 40) {
      ctx.beginPath()
      ctx.moveTo(0, y)
      ctx.lineTo(canvas.width, y)
      ctx.stroke()
    }

    // líneas del blueprint
  }, [points])
}
```

Servicios: apimock y apiclient

1 Se crearon dos servicios con la misma interfaz:

- apimock → devuelve datos falsos guardados en memoria (para probar sin backend).
- blueprintsApi → consume el API real usando Axios.

Ambos tienen los mismos métodos:

getAll
getByAuthor
getByAuthorAndName
create

2 Se creó un servicio intermedio

blueprintsService.js

Este archivo decide cuál usar:

```
import apimock from './apimock.js'  
import apiclient from './blueprintsApi.js'
```

```
const useMock = import.meta.env.VITE_USE MOCK === 'true'
```

```
const service = useMock ? apimock : apiclient
```

```
export default service
```

3 Se controla con una variable en .env

VITE_USE MOCK=true

- true → usa apimock
- false → usa apiclient

4 Redux ya no llama directamente a Axios

Antes:

slice → axios → backend

Ahora:

slice → blueprintsService → (mock o api real)

```

1  import api from './apiClient.js'
2
3  const blueprintsApi = {
4
5    getAll: async () => {
6      const { data } = await api.get('/blueprints')
7      return data.data
8    },
9
10   getByAuthor: async (author) => {
11     const { data } = await api.get(`/blueprints/${encodeURIComponent(author)}`)
12     return data.data
13   },
14
15   getByAuthorAndName: async (author, name) => {
16     const { data } = await api.get(
17       `/blueprints/${encodeURIComponent(author)}/${encodeURIComponent(name)}`
18     )
19     return data.data
20   },
21
22   create: async (blueprint) => {
23     const { data } = await api.post('/blueprints', blueprint)
24     return data
25   },
26 }
27
28 export default blueprintsApi

```

```

import apimock from './apimock.js'
import apiclient from './blueprintsApi.js'

const useMock = import.meta.env.VITE_USE MOCK === 'true'

const service = useMock ? apimock : apiclient

export default service

```

```

const mockData = [
  {
    author: 'juan',
    name: 'house',
    points: [
      { x: 50, y: 50 },
      { x: 120, y: 50 },
      { x: 120, y: 120 },
      { x: 50, y: 120 },
    ],
  },
  {
    author: 'juan',
    name: 'tree',
    points: [
      { x: 200, y: 60 },
      { x: 240, y: 120 },
      { x: 160, y: 120 },
    ],
  },
]

const apimock = {}
  getAll: async () => {
    return mockData
  },

  getByAuthor: async (author) => {
    return mockData.filter((bp) => bp.author === author)
  },

  getByAuthorAndName: async (author, name) => {
    return mockData.find((bp) => bp.author === author && bp.name === name)
  },

  create: async (blueprint) => {
    mockData.push(blueprint)
    return blueprint
  },
}

export default apimock

```

Interfaz con React

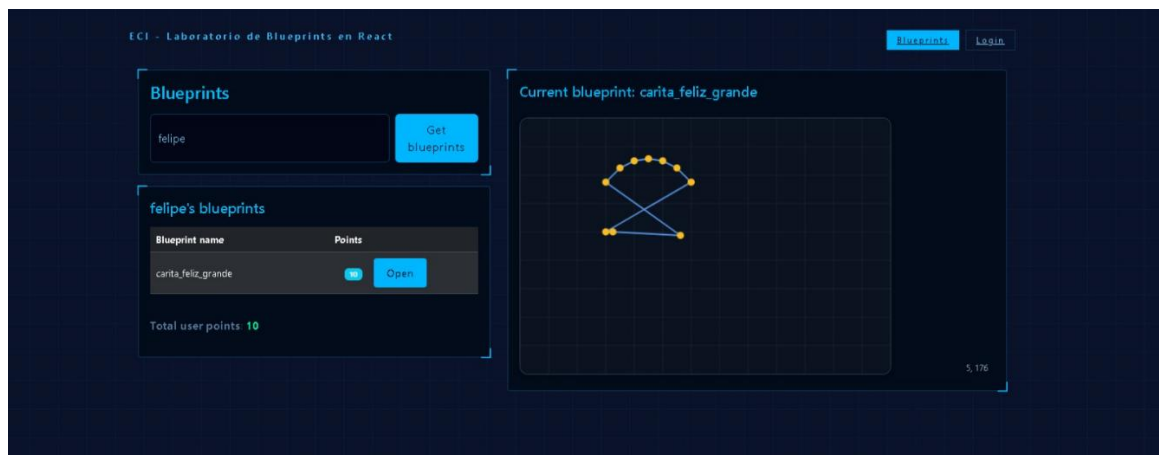
En este punto la interfaz usa React y Redux para manejar el estado de la aplicación. El nombre del plano seleccionado se almacena en el estado global de Redux (current). Cuando el usuario presiona el botón Open, se consulta el plano correspondiente y se actualiza ese estado.

Luego, el componente BlueprintsPage obtiene esa información con useSelector y muestra el nombre del plano actual en el DOM mediante JSX. De esta forma no se

manipula el DOM directamente, sino que React actualiza automáticamente la interfaz cuando cambia el estado. Además, los puntos del plano seleccionado se envían al componente BlueprintCanvas para dibujarlo en el lienzo.

```
const openBlueprint = (bp) => {  
  const token = localStorage.getItem('token')  
  if (!token) return  
  
  dispatch(fetchBlueprint({ author: bp.author, name: bp.name })))  
}  
  
return (  
  <div className="container-fluid">  
    <div className="row g-4">  
  
      { /* LEFT SIDE */ }  
      <div className="col-md-5">
```

Estilos



Para mejorar la presentación de la aplicación se realizó una modificación de los estilos visuales utilizando Bootstrap como framework CSS, junto con estilos personalizados definidos en los archivos CSS del proyecto.

Primero, se integró Bootstrap importando su hoja de estilos en el proyecto de React. Esto permitió utilizar componentes y clases predefinidas para mejorar la organización visual de la interfaz, especialmente en elementos como botones, formularios y contenedores.

- Botones: Se aplicaron estilos para que los botones mantuvieran una apariencia consistente con el diseño del laboratorio, incluyendo colores azulados y bordes definidos.

- Campos de entrada (input): Se ajustaron los estilos del campo de autor para que mantuviera el esquema de colores del tema oscuro de la aplicación, evitando que los estilos por defecto de Bootstrap modificaran su apariencia.
- Tarjetas o contenedores: Se organizaron las secciones de *Blueprints*, *Results* y *Current blueprint* utilizando contenedores tipo tarjeta para mejorar la separación visual de los elementos.
- Distribución de la interfaz: Se estructuró la página en dos columnas principales, donde el panel izquierdo contiene el buscador y los resultados, y el panel derecho muestra el plano actual y el área de dibujo.

Adicionalmente, se ajustaron algunos estilos globales como colores de texto, bordes y fondos para mantener coherencia con el diseño del laboratorio, que utiliza un tema oscuro con detalles en azul.

Pruebas Unitarias

Durante la ejecución del comando `npm test` se corrieron las pruebas automatizadas del proyecto utilizando Vitest junto con React Testing Library y Jest DOM para verificar el funcionamiento de los componentes desarrollados en React. En total se ejecutaron cuatro pruebas que validaron distintos aspectos del sistema: el renderizado del componente de canvas y la llamada al método `getContext`, el correcto envío y parseo de datos en el formulario de blueprints, el despacho de la acción `fetchByAuthor` en la página principal utilizando Redux Toolkit, y la correcta inicialización del estado del reducer de blueprints.

La ejecución finalizó exitosamente, con 4 pruebas pasadas de 4, sin errores ni fallos, en un tiempo aproximado de 3.99 segundos, lo que confirma que los componentes y la lógica básica del módulo funcionan correctamente.

```
PS C:\Users\pacoa\Desktop\Laboratorio-6-ARSW> npm test

RUN v3.2.4 C:/Users/pacoa/Desktop/Laboratorio-6-ARSW

✓ tests/blueprintsSlice.test.jsx (1 test) 8ms
✓ tests/BlueprintCanvas.test.jsx (1 test) 190ms
✓ tests/BlueprintForm.test.jsx (1 test) 228ms
✓ tests/BlueprintsPage.test.jsx (1 test) 274ms

Test Files  4 passed (4)
Tests      4 passed (4)
Start at   18:27:46
Duration   3.99s (transform 687ms, setup 2.29s, collect 3.55s, tests 701ms, environment 5.81s, prepare 907ms)

PS C:\Users\pacoa\Desktop\Laboratorio-6-ARSW>
```

Luego se muestra una ejecución exitosa de un pipeline de integración continua (CI) configurado en GitHub Actions. En este proceso automático se ejecutó el workflow llamado node-ci, el cual se activó tras realizar cambios en el repositorio. El pipeline preparó el entorno instalando Node.js, descargó el código del repositorio mediante actions/checkout, instaló las dependencias del proyecto con npm install, y posteriormente ejecutó varias tareas del proyecto.

